DCLAR: Software Architecture, Program Flow, and State Management

Data Center Leak Assessment Robot
Systems-Level Software Engineering: Shane Canet

October 21, 2026

Contents

1 System Overview **2**
1.1 Software responsibilities . . . 2
1.2 Architecture at a glance . . . 2

2 How the Python Modules Communicate **2**
2.1 Communication model . . . 2
2.2 Module interfaces . . . 3
2.3 Why separate the modules? . . . 3

3 Program Execution Flow **3**
3.1 Startup to mission completion . . . 3
3.2 Control flow versus data flow . . . 4

4 The Map as the Program's Digital Model **4**
4.1 The `SearchGrid` object . . . 4
4.2 Important map-state fields . . . 4
4.3 Grid coordinates . . . 5

5 How Path Data Is Represented **5**
5.1 Map-level route: `planned_route` . . . 5
5.2 Local tile path: `path_points` . . . 5
5.3 Movement command representation . . . 5
5.4 Completed path records . . . 6

6 Coordinate Frames and Heading Conventions **6**

7 Simulation and Graphical User Interface **6**
7.1 Role of the Tkinter callback . . . 6
7.2 Why the event loop matters . . . 6
7.3 Planning versus execution . . . 6

8 Spill Locations and Simulated Leak Events **7**

9 Saving State and Exporting the Plan **7**
9.1 Runtime memory versus a JSON file . . . 7
9.2 What the export represents . . . 7
9.3 Why data format and state ownership matter . . . 7

10 Digital-Twin Scope and System Boundaries **7**

11 Systems-Level Design Decisions **8**
11.1 Separation of responsibilities . . . 8
11.2 Consistent state and coordinate contracts . . . 8
11.3 Current limitations and next engineering steps . . . 8

12 Summary **8**

1 System Overview

The Data Center Leak Assessment Robot (DCLAR) is a software prototype for planning and visualizing a robot's inspection of a gridded data-center floor. The software represents the environment as a collection of tiles, computes routes between inspection targets, generates a local search pattern within each tile, and animates the resulting movement through a graphical interface.

The primary systems-engineering challenge is not an individual algorithm in isolation. It is defining how the map, route planner, local search planner, simulated robot state, and user interface fit together. Each component must use compatible coordinate conventions and exchange data in a predictable form.

The current program is best understood as a **map-based planning and simulation prototype**. It demonstrates software structure and planned movement, but it is not yet a complete robot-control system connected to physical motors, localization hardware, or leak sensors.

1.1 Software responsibilities

The main program is divided into three principal Python modules:

Module	Primary responsibility	System-level role
`main.py`	Coordinates the mission and user interface	Owns the high-level program flow, current simulated robot state, and transition between inspection targets.
`route.py`	Plans movement between map cells	Provides a map-level route to the next target while respecting the grid and obstacles.
`search.py`	Generates a search pattern within a tile	Converts a target tile into a local sequence of points for systematic coverage.

1.2 Architecture at a glance

The main program calls the planning modules, receives their results, and passes those results to the visualization and simulation logic. This is ordinary in-process Python communication: modules expose functions and classes, and other modules import and call them. The current architecture does not require a network connection or a message broker between these scripts.

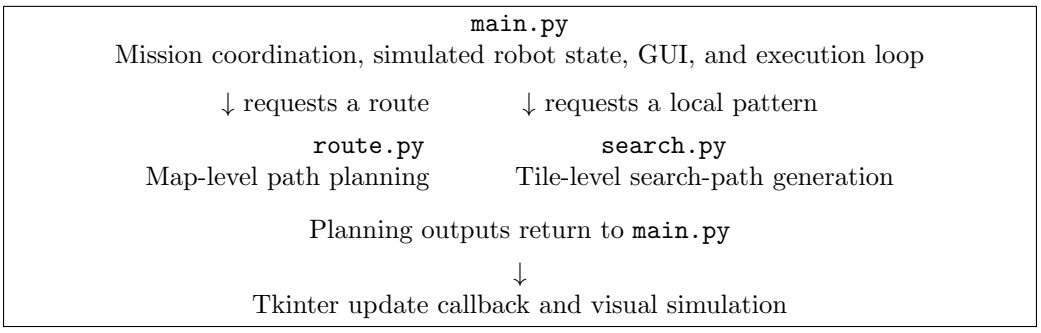


Figure 1: Conceptual software architecture. The main module coordinates the other modules and the graphical simulation.

2 How the Python Modules Communicate

2.1 Communication model

The modules communicate through function calls, arguments, return values, and shared data objects. For example, the coordinator can ask the route planner for a sequence of map cells, then ask the search module for a local pattern for a target tile. The returned Python lists, tuples, and objects are held in memory and used by the main program.

This design is simpler than a distributed architecture: there is no need to serialize each function call into a network message, handle network latency, or synchronize independent processes. The

tradeoff is that the components are currently coupled through Python interfaces and the application runs as one program.

2.2 Module interfaces

The key interfaces are summarized below. The names describe the roles used in the current project; the exact function arguments should be kept aligned with the implementation as it evolves.

- **Route planning:** `path_between(...)` is called to find a route from the robot's current map cell to a target cell. Its output is a sequence of grid coordinates.
- **Local search planning:** `make_search_path(...)` creates a set of local points that cover a target tile. These points are expressed relative to that tile rather than as absolute map coordinates.
- **Map-state updates:** the `SearchGrid` object stores information such as map dimensions, obstacles, searched cells, and the current and target cells. The coordinator and planning routines use this state to make decisions.
- **GUI updates:** the main program schedules an `update_search` callback. The callback advances the simulation and requests another update from Tkinter, allowing the display to animate without a separate network service.

2.3 Why separate the modules?

The split creates clear responsibilities. A change to the local zigzag search pattern can be made in `search.py` without rewriting the mission coordinator. A change to route selection can be isolated in `route.py`, provided its input and output contract remains compatible. The main program still integrates the components and must be updated if an interface or coordinate convention changes.

This is a systems-level design choice: the value comes from defining component boundaries and their contracts, not from treating each source file as an independent application.

3 Program Execution Flow

3.1 Startup to mission completion

At a high level, execution proceeds through the following stages:

1. **Initialize the environment.** Construct or load the grid representation, including its dimensions, obstacle cells, and target information.
2. **Initialize simulated robot state.** Set the robot's current map location and heading, and initialize the path and movement variables used by the animation.
3. **Select the next target.** Determine which inspection tile should be visited next according to the mission logic.
4. **Plan transit.** Call the route planner to generate a map-level route from the current cell to the selected target.
5. **Generate local coverage.** Call the search-pattern generator to obtain a sequence of points within the target tile.
6. **Animate the planned motion.** The Tkinter callback advances the simulated robot along the planned path and updates the graphical display.
7. **Update mission state.** When a tile's search pattern is finished, mark the tile as complete, update the current location, and select the next target if one remains.
8. **Finish or continue.** Continue the plan–execute–update cycle until the mission's target cells have been handled or no further route is available.

3.2 Control flow versus data flow

Control flow describes *which operation happens next*. Data flow describes *which information is passed between operations*. In DCLAR, the control flow is coordinated by `main.py`, while the route and search modules return path data that the coordinator stores and consumes.

For example, the route planner does not need to own the animation timer. Its job is to calculate a route. Likewise, the GUI callback should not independently invent a new inspection strategy; it advances the path selected by the mission logic. Keeping those responsibilities distinct makes the program easier to test and reason about.

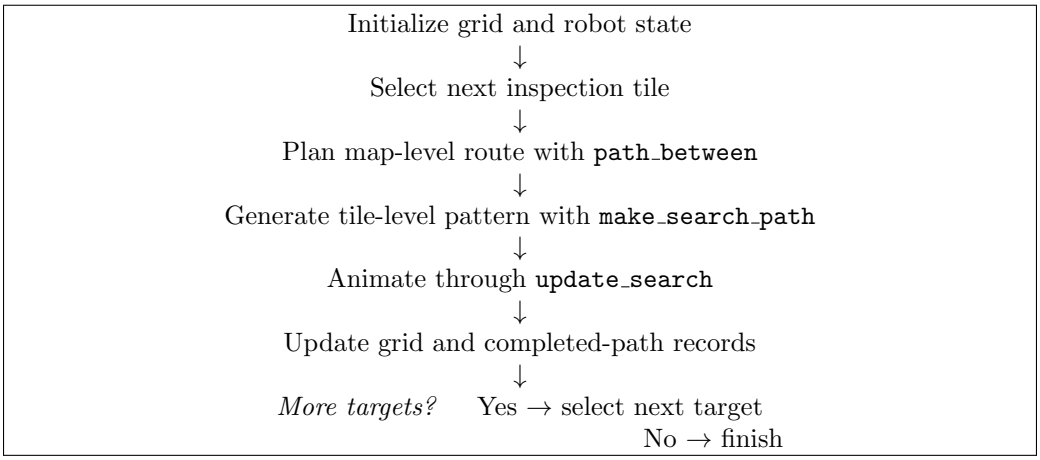


Figure 2: Simplified mission control flow. This is a conceptual flow, not a claim that every step is implemented as a separate function.

4 The Map as the Program's Digital Model

4.1 The `SearchGrid` object

The map is represented in program memory by a grid-oriented state object called `SearchGrid`. Instead of representing the entire facility as a continuous geometric model, the program divides the floor into discrete cells. Each cell has a row and column index, and cells can be associated with states such as obstacle, searched, current, or target.

The grid acts as a shared model of the environment for planning. The route planner can use obstacles to avoid blocked cells, while the mission coordinator can use searched-cell information to decide which inspection target remains.

4.2 Important map-state fields

Field	Meaning
`rows`, `columns`	Number of grid rows and columns. These define the map's discrete extent.
`obstacles`	A collection of cells that the planner should treat as blocked. A set is useful because membership checks are fast and duplicate cells are naturally avoided.
`searched`	A collection of cells already treated as searched by the simulation. This supports mission progress tracking.
`order`	An ordered collection used to represent a sequence of cells or planned visit order. Unlike a set, a list preserves sequence.
`current_cell`	The grid cell currently associated with the simulated robot.
`target_cell`	The grid cell currently selected as the destination or inspection target.

The exact state fields and their update rules should remain consistent across the coordinator and planning modules. If one component changes the coordinate convention or the meaning of a state flag, the other components can silently interpret the map incorrectly.

4.3 Grid coordinates

Map locations are represented as integer pairs, typically `(row, column)`. These are indices, not physical distances. In Python, indexing normally begins at zero, so the first cell is `(0, 0)` rather than `(1, 1)`.

The project uses a one-foot tile convention for the map-level grid. Under the current convention, map column increases toward the right/east and map row increases downward/south. A row-column pair must not be confused with an ordinary Cartesian `(x,y)` pair: in a row-column pair, the first value identifies the vertical grid index and the second identifies the horizontal grid index.

(0,0)	(0,1)	(0,2)	···
(1,0)	(1,1)	(1,2)	···
(2,0)	(2,1)	(2,2)	···

The grid is the planning abstraction, not a guarantee of real-world position accuracy. A physical robot would need localization and mapping to estimate where it is relative to the actual facility.

5 How Path Data Is Represented

DCLAR uses more than one path representation because the system solves two different problems: moving between map tiles and systematically covering the inside of a tile. These representations should be treated as different data contracts rather than interchangeable lists of coordinates.

5.1 Map-level route: `planned_route`

The map-level route is a sequence of grid cells connecting the robot's current cell to a target cell. A conceptual example is

```
planned_route = [
    (2, 1),
    (2, 2),
    (2, 3),
    (3, 3),
]
```

Each pair identifies a map cell. This route describes *which cells to traverse*; it does not by itself specify the detailed sweep pattern within an inspection tile. Transit through a cell also does not automatically mean that the cell's full local inspection pattern has been completed.

5.2 Local tile path: `path_points`

The local search planner produces points relative to a particular tile. In the project's local coordinate convention, a point `(u,v)` describes an offset within the tile, rather than a global map location.

If the tile has map row r and column c, a local point can be mapped to global tile coordinates using the project's stated convention:

$$x = c + u, \qquad y = r + v.$$

This mapping is a coordinate transformation between two reference frames: one attached to the tile and one attached to the full grid. Any implementation that converts local paths to global coordinates must use the same axis directions and units as the visualization and movement logic.

5.3 Movement command representation

The coordinator also maintains movement-command data for the animation. A command can include quantities such as a pivot angle, a global heading, a target position in feet, and a forward distance. This representation is closer to a sequence of motion instructions than a simple list of grid cells.

The distinction matters because a route point, a local search point, and a movement command answer different questions:

- **Route point:** Which grid cell should be visited next?

- **Local path point:** Where should the robot move within the current tile to cover it?
- **Movement command:** What simulated turn and forward movement advances the robot toward the next point?

5.4 Completed path records

After a local path is completed, the program can retain records of the finished pattern and its derived segment geometry. These records support visualization and export of what the simulation planned or completed. They should not be confused with independent evidence that a physical robot actually drove those segments.

6 Coordinate Frames and Heading Conventions

A system that combines grid routing, local coverage, and robot-like movement must define its coordinate conventions explicitly. Otherwise, a path can be correct in one representation but appear rotated, mirrored, or offset when converted to another.

The current simulation conventions are:

- A map tile is nominally one foot across.
- Map x increases to the right/east; map y increases downward/south.
- Heading 0° points east, 90° points south, 180° points west, and 270° points north.
- Positive pivot angles are clockwise.
- Local path points are offsets relative to a tile and must be transformed before they are interpreted as map-level positions.

These conventions are part of the software interface. Changing one convention requires reviewing route conversion, heading calculations, GUI drawing, movement commands, and JSON output together.

7 Simulation and Graphical User Interface

7.1 Role of the Tkinter callback

The GUI uses Tkinter's event loop to update the visualization. The `update_search` callback advances the simulated execution and schedules future updates. This lets the interface remain responsive while the robot icon and path display change over time.

The callback is the bridge between a precomputed plan and its visual playback. It reads the active path or movement-command state, updates the simulated robot's position and heading, updates the drawing, and determines whether the current local path has finished.

7.2 Why the event loop matters

A graphical application normally cannot run an endless blocking loop for animation, because that would prevent the interface from processing redraws and user input. Scheduling repeated callbacks lets Tkinter regain control between simulation updates.

This is still a simulation loop, not a real-time motor-control loop. The callback's timing controls how the animation is displayed; it does not establish that a physical robot could achieve the same speed, turning radius, or timing.

7.3 Planning versus execution

The current software visualizes planned movement and updates simulated state according to the program's path data. It does not establish closed-loop execution using physical sensors. In a real system, the planned route would be only a reference: localization, obstacle sensing, motion control, and fault handling would be needed to determine whether the robot actually followed it.

8 Spill Locations and Simulated Leak Events

The prototype can associate a simulated leak event with a map tile and a local position within that tile. Keeping both values is useful: the map cell identifies the general facility location, while the local coordinates identify the approximate position inside the tile.

The current behavior is a simulation of leak discovery rather than a sensor-driven detection system. A simulated stop can be triggered when the robot comes within the configured distance threshold, noted in the project as 3.5 inches, and the interface can request an acknowledgement. This demonstrates the interaction between a location event, the simulated robot, and the user interface.

A production leak-assessment robot would need to replace or supplement this trigger with a sensor interface, calibrated detection criteria, confidence handling, timestamps, and a policy for false positives or unavailable sensor readings.

9 Saving State and Exporting the Plan

9.1 Runtime memory versus a JSON file

While the application is running, the current map, robot state, planned route, local path, movement commands, and leak annotations are held in Python objects and variables. This in-memory state is what the running program reads and updates.

The project also provides a `save_path_plan` operation that writes a JSON representation to `search_path_plan.json` alongside `main.py`. JSON is a text format for structured data. It can represent dictionaries, lists, strings, numbers, booleans, and null values, making it useful for exporting the current plan in a language-independent form.

9.2 What the export represents

The exported plan can include information such as coordinate conventions, map dimensions, rack layout, completed path records, the planned route, the active local path, the current movement-command vector, and simulated spill annotations or leak events.

This file is best treated as a **snapshot or export of planning state**. Saving a JSON file does not automatically mean the application can restart later and restore the entire mission exactly where it left off. Reliable mission resumption would also require a defined load procedure, validation of the saved schema, restoration of all required state, and explicit handling of incomplete or inconsistent records.

9.3 Why data format and state ownership matter

A clear export schema makes the plan inspectable outside the GUI and creates a possible interface to future tools. However, the in-memory objects remain the active source of state during a run unless the program explicitly reloads the JSON file. The export should therefore not be described as a live database or a continuously synchronized digital twin.

10 Digital-Twin Scope and System Boundaries

The term *digital twin* is sometimes used for a software model that stays synchronized with a physical asset through live data. DCLAR currently has a digital representation of a tiled environment and simulated robot state, but the software described here does not establish a continuous sensor connection to a physical robot.

The current boundary is therefore:

Map and Mission State → Route and Search Planning → Simulated Movement → Visualization and JSON Export.

The future physical-system boundary would need to include real localization, sensor readings, motor commands, acknowledgements, and fault states. These additions should be introduced through explicit interfaces rather than mixed directly into route-generation logic.

11 Systems-Level Design Decisions

The main systems-level engineering work is defining the responsibilities, interfaces, state representation, and operational flow of the software. AI-assisted coding can help produce or revise implementation details, but the resulting system still depends on deliberate architectural decisions and integration checks.

11.1 Separation of responsibilities

The current module split separates high-level coordination, map-level routing, and local coverage planning. This supports iteration because each component can evolve while preserving its input/output contract. The coordinator is responsible for sequencing these pieces and translating their outputs into the simulated mission.

11.2 Consistent state and coordinate contracts

The system depends on several invariants:

1. All map-level locations use the same row-column ordering.
2. Local tile coordinates are transformed using a documented reference frame.
3. Obstacles and searched cells have consistent meanings across planning and visualization.
4. A transit route is not treated as proof that an inspection pattern has been completed.
5. Planned movement and simulated movement are not represented as verified physical execution.
6. Exported JSON uses consistent units and coordinate conventions.

These invariants are important integration requirements. Violating one can produce a program that runs without errors but displays or records an incorrect path.

11.3 Current limitations and next engineering steps

1. **Persistent mission state:** define a versioned JSON schema and implement validated save/load behavior if missions must resume after closing the application.
2. **Plan versus execution records:** distinguish intended paths, simulated progress, and confirmed physical movement.
3. **Event history:** record target selection, path completion, simulated leak events, acknowledgements, and errors with timestamps.
4. **Hardware interfaces:** define clean interfaces for localization, obstacle sensing, leak sensing, and motor control before connecting physical hardware.
5. **Fault handling:** specify behavior when a route cannot be found, a target is unreachable, sensor data is missing, or a movement command fails.
6. **Automated tests:** test route validity, obstacle avoidance, coordinate transformations, local-pattern bounds, state transitions, and JSON serialization.

12 Summary

DCLAR is organized around a central coordinator, a map-level route planner, and a local tile-search planner. The modules communicate through ordinary Python interfaces, while the `SearchGrid` object and related runtime variables hold the active simulated mission state. Path information is represented at several levels: grid cells for transit, local points for tile coverage, movement commands for simulated motion, and completed-path records for reporting.

The JSON export records a representation of the plan, but it is not by itself a live database or a full save-and-resume mechanism. Likewise, the current map and animation are a useful digital model of the inspection process, but they do not establish closed-loop control of a physical robot. The most important next steps are to formalize interfaces and invariants, improve state persistence and event tracking, and define the hardware-facing contracts needed to move from a planning prototype toward a deployable robotic system.